

Resumo de Estudo: Laços e Condicionais em Python

Este resumo foi criado para auxiliar seus estudos para a prova, focando nos conceitos de laços e condicionais em Python, com base nas questões resolvidas.

Tópico 1: Laços `while` vs. `for`

`while` (Enquanto):

- **O que é?** Repete um bloco de código **enquanto** uma condição for verdadeira.
- **Estrutura:**

```
python i = 0 # 1. Inicializa o contador while i < 10: # 2. Define a
condição de parada print(i) i = i + 1 # 3. ATUALIZA o contador
(essencial!)
```

- **Macete:** Use `while` quando você **não sabe** quantas vezes precisa repetir, mas sabe a **condição para parar** (ex: "peça um número até o usuário digitar 0").
- **Cuidado!** Se esquecer de atualizar o contador dentro do laço (o `i = i + 1`), você cria um **loop infinito**! Isso foi crucial na análise da Questão 10.

`for` (Para cada):

- **O que é?** Repete um bloco de código para **cada item** de uma sequência (como uma lista ou um `range`).
- **Estrutura:**

```
python
```

range(1, 5) gera os números 1, 2, 3, 4

```
for numero in range(1, 5): print(numero) `` `
```

- **Macete:** Use `for` quando você **sabe** o número de repetições ou quer percorrer todos os itens de uma lista/sequência. É mais seguro contra loops infinitos.
-

Tópico 2: O Poder do `range()`

O `range()` é seu melhor amigo em laços `for`.

- `range(fim)`: Vai de 0 até `fim - 1`.
 - Ex: `range(5)` -> 0, 1, 2, 3, 4
- `range(inicio, fim)`: Vai de `inicio` até `fim - 1`.
 - Ex: `range(2, 6)` -> 2, 3, 4, 5
- `range(inicio, fim, passo)`: Pula os números de acordo com o `passo`.
 - Ex: `range(0, 10, 2)` -> 0, 2, 4, 6, 8 (só os pares)

Macete da Prova: Para incluir o número final `n` no seu laço, use sempre `range(..., n + 1)`.

Tópico 3: Condicionais e Operadores Lógicos

- `if`, `else`, `elif`: Permitem que seu código tome decisões.
- **Operador Módulo (%)**: Essencial para problemas de divisibilidade.
 - `numero % 2 == 0`: Verifica se `numero` é **par**.
 - `numero % 3 == 0`: Verifica se `numero` é **múltiplo de 3**.
- **Operadores Lógicos:**
 - **and (E)**: **AMBAS** as condições precisam ser verdadeiras.
 - Ex: `if (x > 5) and (x < 10)`: Verdadeiro só se `x` for 6, 7, 8 ou 9.

- **or (OU): PELO MENOS UMA** das condições precisa ser verdadeira.
 - Ex: `if (dia == "sabado") or (dia == "domingo"):` Verdadeiro se for fim de semana.

Macete da Prova: Leia o enunciado com atenção! Se ele disser "isso **E** aquilo", use `and`. Se disser "isso **OU** aquilo", use `or`. A Questão 9 dependia de entender essa diferença.

Resumo das Questões e Dicas Finais

- **Questão de Loop Infinito (Questão 10):** Se o contador (`i`) só é atualizado dentro de um `if` ou `else`, desconfie! É uma armadilha clássica para criar um loop infinito. Sempre garanta que o contador avance em todas as iterações relevantes.
 - **Questão de Pares com Condições (Questão 9):**
 1. Use laços aninhados (`for` dentro de `for`) para gerar os pares (`x, y`).
 2. Para evitar duplicatas e garantir `x <= y`, comece o segundo laço a partir do primeiro: `for y in range(x, n + 1)`.
 3. Traduza as condições do português para o código usando `and` ou `or` corretamente.
 - **Interpretação de Código:** Para entender o que um programa faz, simule sua execução com um número pequeno (ex: `n=5`). Anote os valores das variáveis a cada passo. Isso revela o padrão e ajuda a identificar erros de lógica ou a prever resultados.
-

Autor: Manus AI **Data:** 21 de agosto de 2025